

Access Modifiers in Java

Access modifiers in Java define the scope and visibility of classes, methods, variables, and constructors. They determine who can access specific parts of your code. Proper use of access modifiers helps achieve encapsulation and promotes secure, maintainable code.

Types of Access Modifiers

Java provides four access modifiers:

Modifier	Class	Package	Subclass	World (Other Packages)
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
(default)	✓	✓	✗	✗
private	✓	✗	✗	✗

1. **public**: The member is accessible from **anywhere** in the program.
2. **protected**: The member is accessible within the **same package** and in **subclasses** in other packages.
3. **default** (no modifier): The member is accessible only within the **same package**.
4. **private**: The member is accessible only within the **same class**.

Detailed Explanation of Access Modifiers

1. public

- **Scope**: Global.
- **Use**: Use **public** for methods, variables, or classes that need to be accessible from anywhere.

Example:

```
// File: PublicExample.java
package mypackage;

public class PublicExample {
```

```

        public void displayMessage() {
            System.out.println("Public: Accessible from anywhere!");
        }
    }

// File: TestPublic.java
import mypackage.PublicExample;

public class TestPublic {
    public static void main(String[] args) {
        PublicExample obj = new PublicExample();
        obj.displayMessage(); // Accessible
    }
}

```

2. protected

- **Scope:** Limited to the same package or subclasses in other packages.
- **Use:** Use `protected` for members that should be accessible within their package and by subclasses.

Example:

```

// File: ProtectedExample.java
package mypackage;

public class ProtectedExample {
    protected void displayMessage() {
        System.out.println("Protected: Accessible in package and
subclasses.");
    }
}

// File: TestProtected.java
package mypackage;

public class TestProtected {
    public static void main(String[] args) {
        ProtectedExample obj = new ProtectedExample();
        obj.displayMessage(); // Accessible within the same package
    }
}

// File: SubClassProtected.java
package anotherpackage;

```

```
import mypackage.ProtectedExample;

public class SubClassProtected extends ProtectedExample {
    public static void main(String[] args) {
        SubClassProtected obj = new SubClassProtected();
        obj.displayMessage(); // Accessible in subclass
    }
}
```

3. default (no modifier)

- **Scope:** Limited to the same package.
- **Use:** Use default for members that should only be accessible within the same package.

Example:

```
// File: DefaultExample.java
package mypackage;

class DefaultExample {
    void displayMessage() {
        System.out.println("Default: Accessible only within the same
package.");
    }
}

// File: TestDefault.java
package mypackage;

public class TestDefault {
    public static void main(String[] args) {
        DefaultExample obj = new DefaultExample();
        obj.displayMessage(); // Accessible
    }
}

// File: TestDefaultAnotherPackage.java
package anotherpackage;

import mypackage.DefaultExample;

public class TestDefaultAnotherPackage {
    public static void main(String[] args) {
        // DefaultExample obj = new DefaultExample(); // Error: Not
accessible
    }
}
```

```
}  
}
```

4. private

- **Scope:** Limited to the defining class.
- **Use:** Use `private` for members that should not be accessible outside the class.

Example:

```
// File: PrivateExample.java  
package mypackage;  
  
public class PrivateExample {  
    private void displayMessage() {  
        System.out.println("Private: Accessible only within the class.");  
    }  
  
    public void accessPrivate() {  
        displayMessage(); // Accessible within the class  
    }  
}  
  
// File: TestPrivate.java  
package mypackage;  
  
public class TestPrivate {  
    public static void main(String[] args) {  
        PrivateExample obj = new PrivateExample();  
        // obj.displayMessage(); // Error: Not accessible  
        obj.accessPrivate(); // Indirect access through a public method  
    }  
}
```

Summary Table

Modifier	Scope	Use
<code>public</code>	Everywhere	For members that should be accessible from anywhere.
<code>protected</code>	Same package + subclasses in other	For members that should be accessible within the package and by subclasses.

Modifier	Scope	Use
	packages	
default	Same package only	For members that should only be accessible within the same package.
private	Same class only	For members that should be hidden from outside access and accessible only within the class.
